

Week 12. Automation (Loop), Hook

Hook

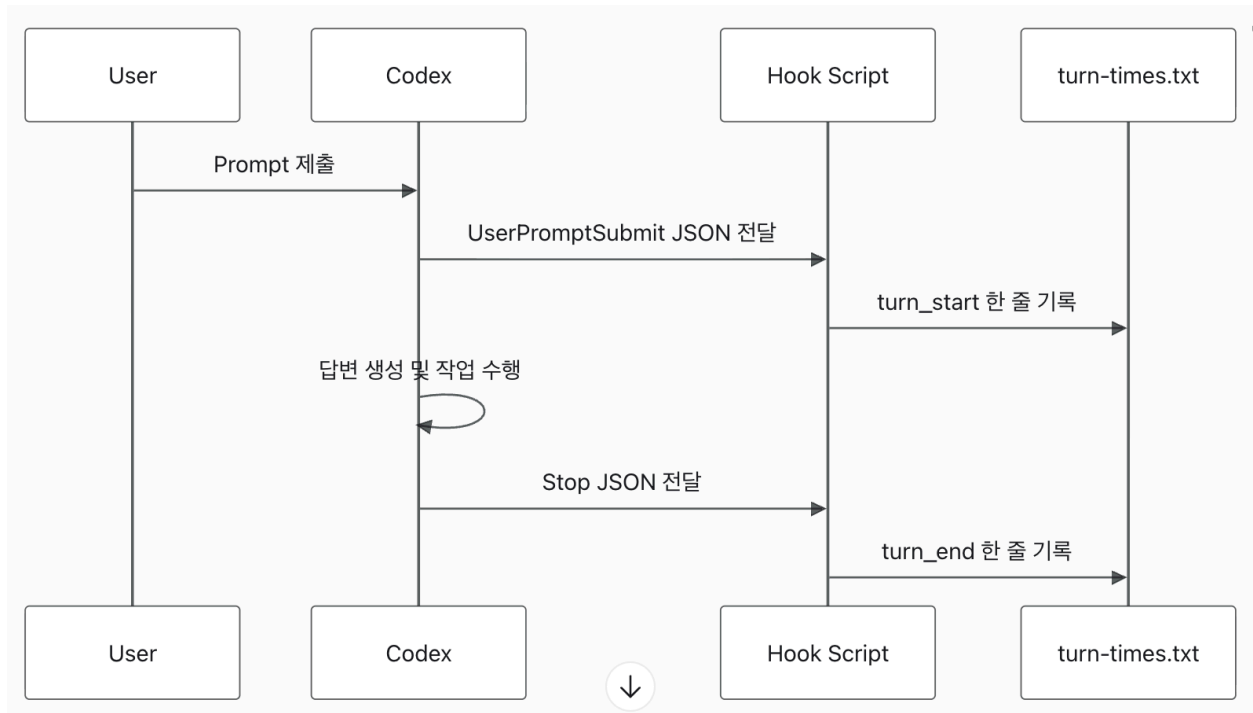
Agent의 생애 주기 동안 특정 시점/조건 만족시 마다 자동으로 실행가능한 스크립트



Events

언제 Hook을 실행할지를 정의

- UserPromptSubmit: 사용자가 프롬프트를 제출했을 때
- PreToolUse: Codex가 도구를 쓰기 직전
- PostToolUse: 도구 실행이 끝난 직후
- PermissionRequest: 권한 요청이 발생할 때
- Stop: 한 turn이 끝나려 할 때
- SessionStart: 세션 시작/재개 시점



접근 방법

```

> /hooks

Hooks
3 hooks configured

[i] This menu is read-only. To add or modify hooks, edit settings.json directly or ask Claude. Learn more

↑ 11. StopFailure    When the turn ends due to an API error
  12. SubagentStart   When a subagent (Agent tool call) is started
  13. SubagentStop    Right before a subagent (Agent tool call) concludes its response
  14. PreCompact      Before conversation compaction
> 15. PostCompact     After conversation compaction

Enter to confirm · Esc to cancel
  
```

Hooks			
Lifecycle hooks from config and enabled plugins.			
Event	Installed	Active	Description
PreToolUse	0	0	Before a tool executes
PermissionRequest	0	0	When permission is requested
PostToolUse	0	0	After a tool executes
PreCompact	0	0	Before context compaction
PostCompact	0	0	After context compaction
SessionStart	0	0	When a new session starts
UserPromptSubmit	0	0	When the user submits a prompt
Stop	0	0	Right before Codex ends its turn

Press enter to view hooks; esc to close

```

Ready (HookPractice) x + v
> /hooks

Security Warning:
Hooks can execute arbitrary commands on your system. Only use hooks from sources you trust. Review hook scripts carefully.

Learn more: https://geminicli.com/docs/hooks

Configured Hooks

BeforeAgent
round-open [enabled]
  Source: project | Command: python .claude/hooks/round_open.py | Matcher: .*

AfterAgent
round-close [enabled]
  Source: project | Command: python .claude/hooks/round_close.py | Matcher: .*

Tip: Use /hooks enable <hook-name> or /hooks disable <hook-name> to toggle individual hooks. Use /hooks enable-all or /hooks disable-all to toggle all hooks at once.

(Press Esc to close)

```

Hook의 유의미한 사용 패턴 (In personal use)

Stochastic Parrot (확률적 앵무새)

Stochastic Parrot은 인공지능 언어모델(LLM)을 비유적으로 설명하기 위해 사용되는 개념입니다. 앵무새가 들은 말을 **이해 없이** 그대로 따라 말하듯, 언어모델도 의미를 '이해'하는 것이 아니라 학습된 **확률에 따라** 다음 단어를 예측하여 생성한다는 비유입니다.

■ 핵심 개념

- LLM은 텍스트의 의미나 진실을 '이해'하지 않는다.
- 오직 학습 데이터에서 단어/토큰의 출현 확률을 통계적으로 학습한다.
- 주어진 맥락에서 다음에 올 단어를 가장 그럴듯한 확률로 예측해 생성한다.
- 그럴듯하지만 틀릴 수 있는(환각, 오류) 답변을 할 수 있다.

■ 예시

사용자: "지구는 왜 동글까?"
모델: (그럴듯한 설명을 생성)
→ 모델은 지구의 물리적 원리를 '이해'해서가 아니라, 학습 데이터에서 이런 질문 다음에 자주 등장하는 단어들을 확률적으로 이어서 말한 것.

■ 한계

- 사실과 다른 내용을 그럴듯하게 말할 수 있음(환각)
- 진실 여부를 스스로 검증하지 못함
- 맥락을 벗어나거나 논리적 일관성이 깨질 수 있음

즉, Stochastic Parrot은 LLM이 '지능'이라기보다 **'확률적 패턴 생성기'**에 가깝다는 점을 강조하는 비유입니다.



AI 생성 이미지(GPT-Image-2)입니다.

At Stop → 에이전트에게 쉘 스크립트 실행, timeout 10s 정도로 정상 exit code 발행되는지 확인시키고 작업 완료 보고하기, 특정 에이전트가 작업 완료시 SDK 혹은 subprocess 등을 활용하여 다른 에이전트 작동 등

At PostToolUse → 어떤 도구 사용했는지 기록 남기기 (LOG)

At UserPromptSubmit → 사용자의 지시에 대해 /docs와 같은 지식 베이스에 이미 존재하는지 체크 시키기, 추가 프롬프트 강제 주입 등

→ 결국 AI Agent가 확률에 따른 잘못된 결정을 내릴 수 있는 부분들을 Hook을 이용하여 결정론적인 결과물을 낼 수 있도록 하는 것이 주 목적이라고 보면 됨

Loop

```

> Codex, 1분마다 "/docs" 폴더에 추가된 파일이 있는지 체크하고, 파일이 추가되면 "/docs/markdown" 폴더에 해당 파일의 마크다운 파싱본을 만들어 주실 수 있나요 ?

• 먼저 C:\AI의 현재 구조와 기존 /docs 규칙을 확인한 뒤, 1분 주기 감시 스크립트와 실행 방식을 맞춰 넣겠습니다.

• Running Get-ChildItem -Force

```

- 해당 기능은 Claude에서만 거의 독점적으로 지원되며, Codex 및 Gemini에서는 OS에서 제공하는 Scheduler 기능을 활용해야 함

예시

```

* Crunched for 3m 24s · 2 shells still running

> /loop 5분 주기로 업로드 진행상황 알려주세요.

● CronCreate(* / 5 * * * *: 업로드 진행상황 알려주세요.)
  L Scheduled 96d52cc1 (Every 5 minutes)

```

- 주의해야 할 점 :
 - 이 작업이 Read / Write 를 수반하는 경우 일정 시간마다 작업 비용을 발생시키므로, 고가의 요금제를 사용하는 것이 아니라면 그림과 같이 단순히 Status Checking 위주로 활용하는 것이 권장됨.
 - 만약 개인 GPU를 이용하여 Qwen3.5 27b 와 같은 로컬 모델을 vllm 혹은 Ollama를 이용하여 호출 할 수 있다면 fire는 상용 LLM 에이전트에게 맡기고, action은 로컬 에이전트를 api로 호출하여 비용 절감을 노릴 수 있음